

✓ Billionaire Statistics Project

✓ Project Overview

The ["Billionaires Statistics Dataset \(2023\)" from Kaggle](#) provides detailed statistics on the world's billionaires, covering aspects such as wealth, business sectors, industries, and personal demographics. Key features of the dataset include wealth metrics (finalWorth), business information (industries), economic indicators (gdp_country, tax_revenue_country_country), and demographic details (age, selfMade, status).

Project interest stemmed from examining if being "Self Made" influences billionaire status and wealth, whether wealth is concentrated among the richest people, the relationship between billionaire wealth and economic factors, and if wealth creation is limited.

This project specifically hones in on the question **"Can we predict whether a billionaire is self-made or has inherited wealth based on their wealth, age, industry, and country-related factors?"**

This question explores the state of economic upward mobility within the countries with the most billionaires. This analysis sheds light on whether wealth accumulation is driven by effort, innovation, and creativity (self-made) compared to inherited privilege. It also considers the proportion of wealth inequality through revealing how wealth is concentrated and passed through generations.

Further questions for analysis, that may utilize different algorithms and non-predictive modeling, include:

- How is wealth distributed among billionaires, and is it concentrated in the hands of a few?
- What percentage of the total wealth is held by the top richest individuals?
- Do billionaires contribute proportionally to national economies, or is wealth hoarded?
- Are most billionaires older, indicating that wealth creation is limited to earlier generations?

✓ Data Selection: Billionaire Statistics

```
!pip install mglearn
import mglearn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
Requirement already satisfied: mglearn in /usr/local/lib/python3.10/dist-packages (0.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from mglearn) (1.26.4)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from mglearn) (3.8.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from mglearn) (1.5)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from mglearn) (2.2.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from mglearn) (11.0.0)
Requirement already satisfied: cycler in /usr/local/lib/python3.10/dist-packages (from mglearn) (0.12.1)
Requirement already satisfied: imageio in /usr/local/lib/python3.10/dist-packages (from mglearn) (2.36.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from mglearn) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->mglearn)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->mglearn)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=
```

Let's begin by taking a look at some of the information the dataset contains and how we can inform our question by looking at and examining the first five rows.

```
bill_stats = pd.read_csv('Billionaires Statistics Dataset.csv') # Read in dataset
bill_stats.head() # Show first 5 rows of dataset
```

	rank	finalWorth	category	personName	age	country	city	source	industries	countryOfCitizenship
0	1	211000	Fashion & Retail	Bernard Arnault & family	74.0	France	Paris	LVMH	Fashion & Retail	France
1	2	180000	Automotive	Elon Musk	51.0	United States	Austin	Tesla, SpaceX	Automotive	United States
2	3	114000	Technology	Jeff Bezos	59.0	United States	Medina	Amazon	Technology	United States
3	4	107000	Technology	Larry Ellison	78.0	United States	Lanai	Oracle	Technology	United States
4	5	106000	Finance & Investments	Warren Buffett	92.0	United States	Omaha	Berkshire Hathaway	Finance & Investments	United States

5 rows x 35 columns

```
bill_stats.shape # Get rows and columns of dataset.
```

```
(2640, 35)
```

Let's look in particular what columns we have that help inform Billionaire Statistics.

```
bill_stats.columns # Show Billionaire Statistics dataset columns
```

```
Index(['rank', 'finalWorth', 'category', 'personName', 'age', 'country',
       'city', 'source', 'industries', 'countryOfCitizenship', 'organization',
       'selfMade', 'status', 'gender', 'birthDate', 'lastName', 'firstName',
       'title', 'date', 'state', 'residenceStateRegion', 'birthYear',
       'birthMonth', 'birthDay', 'cpi_country', 'cpi_change_country',
       'gdp_country', 'gross_tertiary_education_enrollment',
       'gross_primary_education_enrollment_country', 'life_expectancy_country',
       'tax_revenue_country_country', 'total_tax_rate_country',
       'population_country', 'latitude_country', 'longitude_country'],
      dtype='object')
```

Above we note all 35 columns included in the dataset, however, we want to narrow down our focus. In the following data preprocessing section, we begin to pinpoint what columns will help us analyze our question of interest.

✓ *Data Preprocessing*

In this section, we work on data preprocessing. Narrowing in our columns of interest, we find missing values, address them, encode categorical variables using one-hot encoding and apply feature scaling.

✓ *Addressing Missing Values*

We want our analysis to be easy to understand and narrowed into what will be of most value to our question. Considering all of our columns noted earlier, let's pinpoint particular ones of interest and focus our analysis on this area.

Let's begin by creating a function that helps us quickly assess whether one of our columns of interest has a missing value, how many, and what data type it is in order to inform us in our imputation approach.

```
def find_missing(df, columns):
    summary = pd.DataFrame({
        'Column': columns,
        'Missing Values': [df[col].isna().sum() for col in columns],
        'Data Type': [df[col].dtype for col in columns]
    })

    missing_summary = summary[summary['Missing Values'] > 0]

    return missing_summary if not missing_summary.empty else "No missing vals."

interested_cols = [
    "finalWorth", "rank", "age", "selfMade", "status", "category", "country",
    "gdp_country", "tax_revenue_country_country",
    "total_tax_rate_country", "cpi_country", "cpi_change_country"
]
```

I chose "finalWorth", "rank", "age", "selfMade", "status", "category", "country", "gdp_country", "tax_revenue_country_country", "total_tax_rate_country", "cpi_country", and "cpi_change_country" as my initial columns of interest.

The columns represent:

- **finalWorth:** The final net worth of the billionaire in U.S. dollars.
- **rank:** The ranking of the billionaire in terms of wealth.
- **age:** The age of the billionaire.
- **selfMade:** Indicates whether the billionaire is self-made (True/False).
- **status:** "D" represents self-made billionaires (Founders/Entrepreneurs) and "U" indicates inherited or unearned wealth.
- **category:** The category or industry in which the billionaire's business operates.
- **country:** The country in which the billionaire resides.
- **gdp_country:** Gross Domestic Product (GDP) for the billionaire's country.
- **tax_revenue_country_country:** Tax revenue in the billionaire's country.
- **total_tax_rate_country:** Total tax rate in the billionaire's country.
- **cpi_country:** Consumer Price Index (CPI) for the billionaire's country.
- **cpi_change_country:** CPI change for the billionaire's country.

These columns contain information that may reveal connections between economic development, age, wealth hoarding, and selfMade status within billionaires.

```
missing_summary = find_missing(bill_stats, interested_cols)
print(missing_summary)
```

	Column	Missing Values	Data Type
2	age	65	float64
6	country	38	object
7	gdp_country	164	object
8	tax_revenue_country_country	183	float64
9	total_tax_rate_country	182	float64
10	cpi_country	184	float64
11	cpi_change_country	184	float64

Applying the function to our columns of interest, we note that missing values only exist in the columns age, country, gdp_country, tax_revenue_country_country, total_tax_rate_country, cpi_country, and cpi_change_country.

Let's go ahead and copy our original DataFrame to create a new independent DataFrame with our columns of interest to still preserve the original data.

```
bill_stats_interested = bill_stats[interested_cols].copy()
```

This makes it easier to both view and work with our data of interest from now on.

```
bill_stats_interested.head()
```

	finalWorth	rank	age	selfMade	status	category	country	gdp_country	tax_revenue_country_country
0	211000	1	74.0	False	U	Fashion & Retail	France	\$2,715,518,274,227	24.
1	180000	2	51.0	True	D	Automotive	United States	\$21,427,700,000,000	9.
2	114000	3	59.0	True	D	Technology	United States	\$21,427,700,000,000	9.
3	107000	4	78.0	True	U	Technology	United States	\$21,427,700,000,000	9.
4	106000	5	92.0	True	D	Finance & Investments	United States	\$21,427,700,000,000	9.

Next steps:

[Generate code with bill_stats_interested](#)
[View recommended plots](#)
[New interactive sheet](#)

Earlier we saw that, in particular, country and gdp_country are both the only object types with missing values. However, gdp_country can easily be turned into a numerical value. Let's approach this problem first before imputation by removing non-numerical symbols like "\$" and commas. Doing this, we can convert gdp_country to a float64 by utilizing .astype(float).

```
# Clean 'gdp_country' by removing dollar sign and commas
```

```
bill_stats_interested['gdp_country'] = bill_stats_interested['gdp_country'].str.replace('$', '').str.replace(',', '')
```

```
bill_stats_interested.head()
```

	finalWorth	rank	age	selfMade	status	category	country	gdp_country	tax_revenue_country_country	to
0	211000	1	74.0	False	U	Fashion & Retail	France	2.715518e+12		24.2
1	180000	2	51.0	True	D	Automotive	United States	2.142770e+13		9.6
2	114000	3	59.0	True	D	Technology	United States	2.142770e+13		9.6
3	107000	4	78.0	True	U	Technology	United States	2.142770e+13		9.6
4	106000	5	92.0	True	D	Finance & Investments	United States	2.142770e+13		9.6

Next steps:

[Generate code with bill_stats_interested](#)
[View recommended plots](#)
[New interactive sheet](#)

```
bill_stats_interested['gdp_country'].dtype # show gdp_country data type
```

```
↳ dtype('float64')
```

Notice the change in gdp_country data type from categorical object to the now numerical float64.

We can now begin conducting imputation.

Note the following DataFrame table before imputation. The count in each numerical column varies as we have missing values.

```
bill_stats_interested.describe() # before imputation
```

```
↳
```

	finalWorth	rank	age	gdp_country	tax_revenue_country_country	total_tax_rate_country
count	2640.000000	2640.000000	2575.000000	2.476000e+03	2457.000000	2458.000000
mean	4623.787879	1289.159091	65.140194	1.158287e+13	12.546235	43.963344
std	9834.240939	739.693726	13.258098	9.575588e+12	5.368625	12.145296
min	1000.000000	1.000000	18.000000	3.154058e+09	0.100000	9.900000
25%	1500.000000	659.000000	56.000000	1.736426e+12	9.600000	36.600000
50%	2300.000000	1312.000000	65.000000	1.991000e+13	9.600000	41.200000
75%	4200.000000	1905.000000	75.000000	2.142770e+13	12.800000	59.100000
max	211000.000000	2540.000000	101.000000	2.142770e+13	37.200000	106.300000

After identifying what columns missing values are located in in our dataset, we can impute based on their data type.

Numerical columns, like rank and age, will use **mean imputation**, replacing missing values with the average of the non-missing values in the column.

Mean imputation is effective for preserving the distribution of our data while handling numerical values by balancing the properties of our dataset with a representative value.

Categorical columns, such as status and category, will use the **most frequent value imputation (mode)**.

Mode imputation is effective for maintaining consistency in our columns through replacing missing values with common values, reflecting the trend in the data and does not introduce new unknown categories.

```
from sklearn.impute import SimpleImputer

# impute numerical columns with the mean
numerical_cols = ['finalWorth', 'rank', 'age', 'gdp_country', 'tax_revenue_country_country', 'total_tax_rate_country']

imputer_num = SimpleImputer(strategy='mean')
bill_stats_interested[numerical_cols] = imputer_num.fit_transform(bill_stats_interested[numerical_cols])

# impute categorical columns with the most frequent value (mode)
categorical_cols = ['status', 'category', 'country']

imputer_cat = SimpleImputer(strategy='most_frequent')
bill_stats_interested[categorical_cols] = imputer_cat.fit_transform(bill_stats_interested[categorical_cols])
```

Following imputation, all columns now have 2640 rows.

```
bill_stats_interested.describe() # after imputation
```

	finalWorth	rank	age	gdp_country	tax_revenue_country_country	total_tax_rate_country
count	2640.000000	2640.000000	2640.000000	2.640000e+03	2640.000000	2640.000000
mean	4623.787879	1289.159091	65.140194	1.158287e+13	12.546235	43.963344
std	9834.240939	739.693726	13.093803	9.273280e+12	5.179139	11.719011
min	1000.000000	1.000000	18.000000	3.154058e+09	0.100000	9.900000
25%	1500.000000	659.000000	56.000000	1.839758e+12	9.600000	36.600000
50%	2300.000000	1312.000000	65.140194	1.158287e+13	10.100000	43.963344
75%	4200.000000	1905.000000	74.000000	2.142770e+13	12.546235	59.100000
max	211000.000000	2540.000000	101.000000	2.142770e+13	37.200000	106.300000

✓ Encoding Categorical Variables: One Hot Encoding

Let's now explore the categorical variables.

```
bill_stats_interested[categorical_cols]
```

	status	category	country
0	U	Fashion & Retail	France
1	D	Automotive	United States
2	D	Technology	United States
3	U	Technology	United States
4	D	Finance & Investments	United States
...	...	...	...
2635	D	Healthcare	China
2636	E	Food & Beverage	United States
2637	R	Manufacturing	China
2638	D	Real Estate	China
2639	R	Diversified	Philippines

2640 rows × 3 columns

Above, we note that status, category, and country are object types. However, country seems like it would have many distinct values, complicating our analysis and potentially overfitting to countries with little data points.

Let's verify this.

```
print(bill_stats_interested['country'].value_counts())
```

country	
United States	792
China	523
India	157
Germany	102
United Kingdom	82
...	

```

Portugal      1
Georgia       1
Eswatini (Swaziland) 1
Uzbekistan    1
Armenia        1
Name: count, Length: 78, dtype: int64

```

```
print(bill_stats_interested['country'].value_counts().tail(25))
```

```

country
Bermuda      2
Morocco      2
Peru         2
Bahamas      2
New Zealand  2
Turks and Caicos Islands 1
Nepal        1
Uruguay      1
Tanzania     1
Andorra      1
Bahrain      1
Colombia     1
Liechtenstein 1
Guernsey     1
Oman         1
Cambodia     1
British Virgin Islands 1
Luxembourg   1
Latvia       1
Algeria      1
Portugal     1
Georgia      1
Eswatini (Swaziland) 1
Uzbekistan   1
Armenia      1
Name: count, dtype: int64

```

Above, we see that this is the case, we have 78 different values with many only having 1 value in the dataset. Let's narrow in our analysis by focusing on the top 5 most common countries.

```

top_countries = bill_stats_interested['country'].value_counts().nlargest(5).index # selecting only the 5 countries
bill_stats_interested['country_top'] = bill_stats_interested['country'].apply(lambda x: x if x in top_countries else None)

```

Let's drop our original country column now as we no longer need it for future analysis as we will now use country_top.

```

bill_stats_interested.drop(columns=['country'], inplace=True)
bill_stats_interested.head()

```

```

finalWorth  rank  age  selfMade  status  category  gdp_country  tax_revenue_country_country  total_tax_r
0      211000.0    1.0  74.0     False    U  Fashion & Retail  2.715518e+12             24.2
1      180000.0    2.0  51.0      True    D  Automotive  2.142770e+13             9.6
2      114000.0    3.0  59.0      True    D  Technology  2.142770e+13             9.6
3      107000.0    4.0  78.0      True    U  Technology  2.142770e+13             9.6
4      106000.0    5.0  92.0      True    D  Finance & Investments  2.142770e+13             9.6

```

Next steps:

[Generate code with bill_stats_interested](#)
[View recommended plots](#)
[New interactive sheet](#)

We see a similar case occurring within the category column. To reduce model complexity and sparsity, we can instead narrow down categories by combining less frequent categories into an "Other" category. We will identify this as categories with fewer than 100 entries.

```
bill_stats_interested['category'].value_counts()
```



	count
category	
Finance & Investments	372
Manufacturing	324
Technology	314
Fashion & Retail	266
Food & Beverage	212
Healthcare	201
Real Estate	193
Diversified	187
Energy	100
Media & Entertainment	91
Metals & Mining	74
Automotive	73
Service	53
Construction & Engineering	45
Logistics	40
Sports	39
Telecom	31
Gambling & Casinos	25

dtype: int64

```
small_categories = bill_stats_interested['category'].value_counts()[bill_stats_interested['category'].value_co
bill_stats_interested['category_top'] = bill_stats_interested['category'].apply(lambda x: x if x not in small_
```

Let's also drop the original category column now as we no longer need it for future analysis as we will now use category_top.

```
bill_stats_interested.drop(columns=['category'], inplace=True)
bill_stats_interested.head()
```

	finalWorth	rank	age	selfMade	status	gdp_country	tax_revenue_country_country	total_tax_rate_country
0	211000.0	1.0	74.0	False	U	2.715518e+12	24.2	60.7
1	180000.0	2.0	51.0	True	D	2.142770e+13	9.6	36.6
2	114000.0	3.0	59.0	True	D	2.142770e+13	9.6	36.6
3	107000.0	4.0	78.0	True	U	2.142770e+13	9.6	36.6
4	106000.0	5.0	92.0	True	D	2.142770e+13	9.6	36.6

Next steps:

[Generate code with bill_stats_interested](#)[View recommended plots](#)[New interactive sheet](#)

Let's replace these new columns now in our previous categorical_cols subset to remove the original category and country columns.

```
# Replace 'country' and 'category' with 'country_top' and 'category_top' in categorical_cols
categorical_cols = [col for col in categorical_cols if col not in ['country', 'category']]
categorical_cols.extend(['country_top', 'category_top'])
```

```
print(categorical_cols)
```

```
['status', 'country_top', 'category_top']
```

We can now one-hot encode our categorical columns. We do this to transform our data into a way that is easily recognizable for a machine learning algorithm to process and utilize as they often cannot easily manage categorical data.

In One-Hot Encoding, a new binary column is created for each categorical column, a vital step to begin predictive modeling.

This method ensures the algorithm interprets data appropriately, preserves our information, and allows for compatability amongst algorithms.

```
from sklearn.preprocessing import OneHotEncoder
```

```
# ENCODE CATEGORICAL COLUMNS
```

```
encoder = OneHotEncoder(drop='first', sparse_output=False)
```

```
encoded_cats = pd.DataFrame(encoder.fit_transform(bill_stats_interested[categorical_cols]))
```

```
# Add encoded column names based on original columns
```

```
encoded_cats.columns = encoder.get_feature_names_out(categorical_cols)
```

```
# Concatenate the encoded columns back to the preprocessed data
```

```
bill_stats_interested = pd.concat([bill_stats_interested, encoded_cats], axis=1)
```

```
# Drop the original categorical columns
```

```
bill_stats_interested.drop(columns=categorical_cols, inplace=True)
```

```
# Verify bill_stats_interested contains only interested cols
```

```
print(bill_stats_interested.head())
```

	finalWorth	rank	age	selfMade	gdp_country	tax_revenue_country_country	total_tax_rate_country	cpi_country
0	211000.0	1.0	74.0	False	2.715518e+12	24.2	60.7	110.05
1	180000.0	2.0	51.0	True	2.142770e+13	9.6	36.6	117.24
2	114000.0	3.0	59.0	True	2.142770e+13			
3	107000.0	4.0	78.0	True	2.142770e+13			
4	106000.0	5.0	92.0	True	2.142770e+13			

2	9.6	36.6	117.24
3	9.6	36.6	117.24
4	9.6	36.6	117.24

	cpi_change_country	status_E	...	country_top_United States	\
0	1.1	0.0	...		0.0
1	7.5	0.0	...		1.0
2	7.5	0.0	...		1.0
3	7.5	0.0	...		1.0
4	7.5	0.0	...		1.0

	category_top_Energy	category_top_Fashion & Retail	\
0	0.0	1.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	

	category_top_Finance & Investments	category_top_Food & Beverage	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	1.0	0.0	

	category_top_Healthcare	category_top_Manufacturing	category_top_Other	\
0	0.0	0.0	0.0	
1	0.0	0.0	1.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	category_top_Real Estate	category_top_Technology
0	0.0	0.0
1	0.0	0.0
2	0.0	1.0
3	0.0	1.0
4	0.0	0.0

[5 rows x 28 columns]

bill_stats_interested.columns

```

Index(['finalWorth', 'rank', 'age', 'selfMade', 'gdp_country',
      'tax_revenue_country_country', 'total_tax_rate_country', 'cpi_country',
      'cpi_change_country', 'status_E', 'status_N', 'status_R',
      'status_Split Family Fortune', 'status_U', 'country_top_Germany',
      'country_top_India', 'country_top_Other', 'country_top_United Kingdom',
      'country_top_United States', 'category_top_Energy',
      'category_top_Fashion & Retail', 'category_top_Finance & Investments',
      'category_top_Food & Beverage', 'category_top_Healthcare',
      'category_top_Manufacturing', 'category_top_Other',
      'category_top_Real Estate', 'category_top_Technology'],
      dtype='object')

```

A resulting KeyError in the following code block is good. It means we properly removed the original categorical columns and verify that none remain as intended. It is currently commented out to keep the notebook running.

```
# bill_stats_interested[categorical_cols]
```

Below we see our new encoded columns.

encoded_cats.columns

```

Index(['status_E', 'status_N', 'status_R', 'status_Split Family Fortune',
      'status_U', 'country_top_Germany', 'country_top_India',
      'country_top_Other', 'country_top_United Kingdom',

```

```
'country_top_United States', 'category_top_Energy',
'category_top_Fashion & Retail', 'category_top_Finance & Investments',
'category_top_Food & Beverage', 'category_top_Healthcare',
'category_top_Manufacturing', 'category_top_Other',
'category_top_Real Estate', 'category_top_Technology'],
dtype='object')
```

We confirm that all values are now numerical, except for selfMade which is a boolean type for classification we will use in our modeling next steps.

bill_stats_interested.dtypes



0

finalWorth	float64
rank	float64
age	float64
selfMade	bool
gdp_country	float64
tax_revenue_country_country	float64
total_tax_rate_country	float64
cpi_country	float64
cpi_change_country	float64
status_E	float64
status_N	float64
status_R	float64
status_Split Family Fortune	float64
status_U	float64
country_top_Germany	float64
country_top_India	float64
country_top_Other	float64
country_top_United Kingdom	float64
country_top_United States	float64
category_top_Energy	float64
category_top_Fashion & Retail	float64
category_top_Finance & Investments	float64
category_top_Food & Beverage	float64
category_top_Healthcare	float64
category_top_Manufacturing	float64
category_top_Other	float64
category_top_Real Estate	float64
category_top_Technology	float64

dtype: object

✓ Applying Feature Scaling

Our next step is applying feature scaling to our numerical columns. This is an essential step to create consistency among our numerical values. We want to make sure no column dominates our algorithm and leads to biased models, improving our overall performance.

Standard Scaler transforms each feature to have a mean of 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler
```

```
print("Data before scaling:")
print(bill_stats[numerical_cols])
```

```
# Using Standard Scaler: mean of 0 and a standard deviation of 1. Useful in algorithms like logistic regression
scaler = StandardScaler()
bill_stats_interested[numerical_cols] = scaler.fit_transform(bill_stats_interested[numerical_cols])
```

```
print("\nData after scaling:")
print(bill_stats_interested[numerical_cols])
```

```
2639          14.0          43.1          129.61
cpi_change_country
0          1.1
1          7.5
2          7.5
3          7.5
4          7.5
...         ...
2635        2.9
2636        7.5
2637        2.9
2638        2.9
2639        2.5
```

[2640 rows x 8 columns]

Data after scaling:

	finalWorth	rank	age	gdp_country	\
0	20.989450	-1.741806	0.676769	-0.956408	
1	17.836602	-1.740454	-1.080120	1.061835	
2	11.124085	-1.739102	-0.469028	1.061835	

```

2          0.897365
3          0.897365
4          0.897365
...
2635      -0.418994
2636          0.897365
2637      -0.418994
2638      -0.418994
2639      -0.533460

```

[2640 rows x 8 columns]

✓ Feature Engineering (Optional)

Decided to not feature engineer

✓ Model Building

The next step is to use supervised machine learning algorithms to predict whether a billionaire is self-made or has inherited wealth based on our information.

We do this by training and evaluating models, comparing performance of different algorithms, and interpreting the results.

Given the binary target variable "selfMade" we approach our problem using classification algorithms like Logistic Regression, Random Forest, and Gradient Boosting.

✓ Logistic Regression

Supervised Learning Algorithm #1

- Logistic Regression is very effective for binary classification problems.
- Chosen due to usefulness in determining the relationship between categorical outcomes such as *selfMade* and predictors such as *country* and *age*.
- It helps us interpret the impact of each predictor on the likelihood of a billionaire being selfMade through coefficients.
- It is a simple model to begin with and then compare with the more advanced algorithms that follow.

bill_stats_interested.columns

```

Index(['finalWorth', 'rank', 'age', 'selfMade', 'gdp_country',
      'tax_revenue_country_country', 'total_tax_rate_country', 'cpi_country',
      'cpi_change_country', 'status_E', 'status_N', 'status_R',
      'status_Split Family Fortune', 'status_U', 'country_top_Germany',
      'country_top_India', 'country_top_Other', 'country_top_United Kingdom',
      'country_top_United States', 'category_top_Energy',
      'category_top_Fashion & Retail', 'category_top_Finance & Investments',
      'category_top_Food & Beverage', 'category_top_Healthcare',
      'category_top_Manufacturing', 'category_top_Other',
      'category_top_Real Estate', 'category_top_Technology'],
      dtype='object')

```

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score

```

Features and target

```

X = bill_stats_interested.drop(columns=['selfMade'])
y = bill_stats_interested['selfMade'].astype(int)

```

Split data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Logistic Regression
```

```
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train, y_train)
```

```
# Evaluate
```

```
y_pred = log_reg.predict(X_test)
y_pred_proba = log_reg.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_pred_proba):.3f}")
```



	precision	recall	f1-score	support
0	0.65	0.46	0.54	166
1	0.78	0.89	0.83	362
accuracy			0.75	528
macro avg	0.72	0.67	0.68	528
weighted avg	0.74	0.75	0.74	528

AUC-ROC: 0.810

✓ Random Forest

Supervised Learning Algorithm #2

- Random Forest is great to help understand complex relationships between features and the target variable *selfMade*.
- It can identify the most important predictors.
- Provides insight into how being selfMade impacts wealth accumulation.
- Can rank features by importance, letting us identify the most influential factors to wealth and how it differs based on being selfMade or not.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestClassifier
```

```
# Features and target
```

```
X = bill_stats_interested.drop(columns=['selfMade'])
y = bill_stats_interested['selfMade']
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Random Forest Regression
```

```
rf = RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)
rf.fit(X_train, y_train)
```

```
# Evaluate
```

```
y_pred = rf.predict(X_test)
y_pred_proba = rf.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_pred_proba):.3f}")
```

```
feature_importances = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf.feature_importances_
}).sort_values(by='Importance', ascending=False)

print(feature_importances)
```



	precision	recall	f1-score	support
False	0.64	0.57	0.61	166
True	0.81	0.85	0.83	362
accuracy			0.77	528
macro avg	0.73	0.71	0.72	528
weighted avg	0.76	0.77	0.76	528

AUC-ROC: 0.835

	Feature	Importance
2	age	0.194032
1	rank	0.147717
0	finalWorth	0.143771
4	tax_revenue_country_country	0.058194
5	total_tax_rate_country	0.054555
3	gdp_country	0.051883
6	cpi_country	0.048501
7	cpi_change_country	0.042479
26	category_top_Technology	0.032612
12	status_U	0.028870
24	category_top_Other	0.019779
20	category_top_Finance & Investments	0.019516
21	category_top_Food & Beverage	0.019393
19	category_top_Fashion & Retail	0.016571
8	status_E	0.015912
23	category_top_Manufacturing	0.015178
11	status_Split Family Fortune	0.014655
25	category_top_Real Estate	0.013582
22	category_top_Healthcare	0.012874
18	category_top_Energy	0.009971
9	status_N	0.009661
15	country_top_Other	0.007568
17	country_top_United States	0.006571
13	country_top_Germany	0.004922
14	country_top_India	0.004885
10	status_R	0.004561
16	country_top_United Kingdom	0.001789

✓ Gradient Boosting

Supervised Learning Algorithm #3

- Gradient Boosting can improve the performance of Random Forest further.
- Can handle complex relationships while showcasing high accuracy.
- Like Random Forest, it can provide insights into feature importance.
- Is flexible, combines the best of decision trees and ensemble learning in order to predict selfMade status in connection with wealth characteristics.

```
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(
    n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
)
gb.fit(X_train, y_train)

y_pred = gb.predict(X_test)
y_pred_proba = gb.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_pred_proba):.3f}")

feature_importances = pd.DataFrame({
    'Feature': X.columns,
    'Importance': gb.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```
print(feature_importances)
```

		precision	recall	f1-score	support
	False	0.65	0.40	0.50	166
	True	0.77	0.90	0.83	362
	accuracy			0.74	528
	macro avg	0.71	0.65	0.66	528
	weighted avg	0.73	0.74	0.72	528

AUC-ROC: 0.831

	Feature	Importance
4	tax_revenue_country_country	0.183271
2	age	0.135279
3	gdp_country	0.111220
26	category_top_Technology	0.109532
5	total_tax_rate_country	0.091488
6	cpi_country	0.085012
7	cpi_change_country	0.043248
11	status_Split Family Fortune	0.036801
1	rank	0.035846
20	category_top_Finance & Investments	0.033140
0	finalWorth	0.029576
21	category_top_Food & Beverage	0.024903
13	country_top_Germany	0.014526
19	category_top_Fashion & Retail	0.013385
12	status_U	0.012747
18	category_top_Energy	0.009039
17	country_top_United States	0.006726
25	category_top_Real Estate	0.004560
9	status_N	0.004546
8	status_E	0.004436
10	status_R	0.003804
22	category_top_Healthcare	0.003093
14	country_top_India	0.001964
23	category_top_Manufacturing	0.001014
24	category_top_Other	0.000843
15	country_top_Other	0.000000
16	country_top_United Kingdom	0.000000

✓ *Hyperparameter Tuning*

Hyperparameter tuning is a crucial next step in our analysis.

By selecting the most important and optimal hyperparameters, we can greatly improve our models' performances.

✓ *Parameters Chosen*

✓ Logistic Regression

Start coding or [generate](#) with AI.

✓ Random Forest

Start coding or [generate](#) with AI.



```

KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-36-3d407cce716d> in <cell line: 18>()
    16     n_jobs=-1
    17 )
--> 18 grid_search_rf.fit(X_train, y_train)
    19
    20 print(f"Best Parameters: {grid_search_rf.best_params}")

```

7 frames

```

/usr/local/lib/python3.10/dist-packages/joblib/parallel.py in _retrieve(self)
    1760     (self._jobs[0].get_status(
    1761         timeout=self.timeout) == TASK_PENDING)):
-> 1762     time.sleep(0.01)
    1763     continue
    1764

```

KeyboardInterrupt:

▼ Gradient Boosting

Start coding or [generate](#) with AI.

▼ Results and Comparisons

Logistic Regression

Convert results to DataFrame

```

results_lr = pd.DataFrame(grid_search_lr.cv_results_)
results_lr = results_lr[['mean_test_score', 'std_test_score', 'param_C',
'param_penalty', 'param_solver']]

```

Sort by mean test score

```
print(results_lr.sort_values(by='mean_test_score', ascending=False))
```

Random Forest

Convert results to DataFrame

```

results_rf = pd.DataFrame(grid_search_rf.cv_results_)
results_rf = results_rf[['mean_test_score', 'std_test_score',
'param_n_estimators', 'param_max_depth', 'param_min_samples_split']]

```

Sort by mean test score

```
print(results_rf.sort_values(by='mean_test_score', ascending=False))
```

▼ Selected Parameter

Logistic Regression

Evaluate best model

```

y_pred_lr = best_model_lr.predict(X_test)
y_pred_proba_lr = best_model_lr.predict_proba(X_test)[: , 1]

```

from sklearn.metrics import classification_report, roc_auc_score

```
print("Classification Report (Logistic Regression):")
```

```
print(classification_report(y_test, y_pred_lr))
```

```
print(f"AUC-ROC: {roc_auc_score(y_test, y_pred_proba_lr):.3f}")
```

```
# Random Forest
# Evaluate best model
y_pred_rf = best_model_rf.predict(X_test)
y_pred_proba_rf = best_model_rf.predict_proba(X_test)[:, 1]

from sklearn.metrics import classification_report, roc_auc_score
print("Classification Report (Random Forest):")
print(classification_report(y_test, y_pred_rf))
print(f"AUC-ROC: {roc_auc_score(y_test, y_pred_proba_rf):.3f}")
```

✓ *Model's Accuracy and Cross-Validation*

As a classification problem, we report the Accuracy, Precision, recall, F1 score. and Area Under the Receiver Operating Characteristic Curve (AUC-ROC) of our models and compare them.

Furthermore, we implement cross-validation to assess the robustness of each model.

***Summarize the performance of your models with detailed observations and interpretations.

Document your tuning and validation processes, showing code, parameters, and outcomes.

```
from sklearn.metrics import classification_report, roc_auc_score

# Predictions
y_pred = log_reg.predict(X_test)

# Metrics
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, log_reg.predict_proba(X_test)[:, 1]):.3f}")

from sklearn.model_selection import cross_val_score

scores = cross_val_score(log_reg, X, y, cv=5, scoring='accuracy')
print(f"Cross-Validation Accuracy: {scores.mean():.3f} ± {scores.std():.3f}")
```

✓ *Analysis of Results*

Interpret the model results, noting any trends, limitations, and relevant social science insights.